

Relatório Lab 2 - MC714

Aluno: Lucca Chinelato

RA: 239060

1. Descrição do Problema e Contextualização Teórica

No desenvolvimento de sistemas distribuídos, não ter uma memória compartilhada global e de um relógio físico unificado impõe desafios críticos para a coordenação de processos independentes. O objetivo central deste trabalho prático consiste no projeto e na implementação de um ambiente distribuído composto por múltiplos nós independentes que interagem estritamente por meio de troca de mensagens em rede. Para assegurar a consistência global do sistema, tornou-se mandatório resolver três problemas clássicos de sistemas distribuídos: o ordenamento causal e lógico de eventos globais, o acesso ordenado e exclusivo a uma região crítica compartilhada e a resiliência e alta disponibilidade da topologia através da eleição automatizada de um coordenador ou líder de execução.

2. Fundamentação dos Algoritmos Selecionados

Para capturar a relação de precedência causal *happened-before* sem depender de relógios físicos baseados em hardware, foi definido o uso do Relógio Lógico de Lamport. Com esse mecanismo, cada processo mantém um contador incremental local que obedece a duas regras fundamentais de atualização. Antes de registrar um evento interno ou de transmitir uma mensagem para a rede, o nó incrementa seu próprio contador em uma unidade. No momento em que uma mensagem remota é recebida contendo um timestamp associado, o nó destinatário atualiza seu contador local aplicando a função que extrai o valor máximo entre o relógio atual e o recebido, adicionando uma unidade ao final da operação. Essa mecânica garante um ordenamento parcial consistente de todos os eventos do sistema.

Para o gerenciamento do acesso à região crítica, optou-se pelo algoritmo de Ricart-Agrawala. Este protocolo elimina a necessidade de uma trava centralizada, baseando-se no consenso mútuo de todos os pares. Quando uma instância deseja entrar na região crítica, ela altera seu estado interno, captura o timestamp corrente fornecido pelo Relógio de Lamport e dispara uma mensagem de requisição para todos os demais nós ativos da rede. O processo solicitante permanece bloqueado até que receba uma mensagem de resposta afirmativa de todos os participantes. Ao interceptar uma requisição externa, um nó vizinho concede a liberação imediatamente se não estiver interessado na região crítica ou se sua prioridade for menor. Caso o nó local possua maior prioridade e também dispute o recurso, ele

adia a resposta em uma fila interna, liberando-a apenas após concluir sua própria execução crítica. O desempate de prioridades ocorre de forma determinística avaliando o par ordenado composto pelo timestamp lógico e pelo identificador numérico do nó.

Com o intuito de restabelecer a coordenação do sistema na ocorrência de falhas, o algoritmo do Bully foi integrado para a eleição de líder. Este algoritmo assume que cada componente possui um identificador numérico fixo e exclusivo. Quando um nó detecta a inatividade do coordenador atual, ele inicia uma análise enviando uma mensagem de eleição para todos os nós que possuem um identificador estritamente maior que o seu. Se nenhum componente de ID superior responder dentro de um intervalo de tempo limite parametrizado, o nó assume a liderança e transmite uma mensagem de coordenação via *broadcast* para toda a topologia. Por outro lado, se um nó de maior identificador responder com uma mensagem de confirmação, este assume o controle do processo de escolha e o nó inicial passa a aguardar de forma passiva a consolidação do novo líder.

3. Detalhes de Implementação e Concorrência

O ecossistema distribuído foi totalmente desenvolvido em Python, abstendo-se de simulações abstratas em arquivos locais para utilizar conexões de Sockets TCP/IP puras por meio da biblioteca padrão. O tráfego de dados foi padronizado utilizando mensagens estruturadas serializadas em formato JSON. Cada pacote possui campos explícitos para o tipo de mensagem, identificador do remetente, o timestamp do Relógio de Lamport e um dicionário de dados flexível para o payload. Para solucionar a característica de fluxo contínuo do protocolo TCP e mitigar o problema de fragmentação ou colagem de pacotes na camada de transporte, implementou-se um cabeçalho de tamanho fixo contendo a extensão do payload em bytes, empacotado na rede com o auxílio da biblioteca nativa struct.

A arquitetura interna de cada nó opera de forma assíncrona recorrendo a múltiplos fluxos de execução paralela controlados pela biblioteca de threading. A thread servidora principal opera em modo de escuta na porta TCP associada ao nó, delegando o atendimento de cada nova conexão aceita a uma thread de despacho temporária. Ao mesmo tempo, uma thread gerencia a interface de linha de comando interativa do usuário de maneira não bloqueante. Para evitar condições de corrida decorrentes do acesso simultâneo a variáveis de estado compartilhadas, como as filas de mensagens adiadas do Ricart-Agrawala e os contadores do relógio lógico, mecanismos de exclusão mútua local baseados em travas atômicas do tipo Lock foram aplicados.

4. Cenários de Testes e Validação Prática

Os experimentos práticos de comunicação em rede foram validados em um cenário composto por quatro nós ativos executados em diferentes terminais, utilizando conexões TCP/IP. O tráfego base foi avaliado através do envio de mensagens genéricas de teste (Opção 2) entre as instâncias operacionais. Ao disparar um pacote, o nó remetente incrementou o seu contador local e transmitiu os dados estruturados em JSON via socket. Na recepção, a thread servidora do nó de destino realizou a leitura exata dos bytes através do cabeçalho fixo, extraiu o payload e atualizou o seu Relógio de Lamport aplicando a função de valor máximo acrescido de uma unidade, demonstrando a correta sincronização lógica dos eventos no sistema distribuído.

No cenário de exclusão mútua distribuída, simulou-se a disputa concorrente acionando as requisições de acesso (Opção 3) simultaneamente nas interfaces do Nó 1 e do Nó 2. Ambos os processos transmitiram mensagens do tipo REQUEST contendo os seus respectivos timestamps para todos os outros nós. O Nó 1, por possuir um timestamp menor ou menor ID no critério de desempate, coletou os pacotes do tipo REPLY de todos os participantes e acedeu à região crítica. O Nó 2, ao interceptar o REQUEST do Nó 1 e computar que a sua própria prioridade era inferior, adiou a sua resposta numa fila interna. O acesso do Nó 2 permaneceu bloqueado até o momento em que o utilizador acionou manualmente a liberação da região crítica no Nó 1 (Opção 4), o que engatilhou o envio imediato do REPLY pendente via socket, permitindo ao Nó 2 assumir a exclusão mútua de forma segura.

A validação do algoritmo do Bully foi realizada de forma controlada através do acionamento manual do escrutínio no menu interativo (Opção 5), mantendo todos os componentes em execução. Ao disparar o processo a partir da interface do Nó 2, o componente transmitiu imediatamente mensagens do tipo ELECTION em rede apenas para os nós com identificadores superiores, correspondendo ao Nó 3 e ao Nó 4. Ao receber o pacote, o Nó 3 devolveu uma mensagem do tipo OK para o Nó 2 via socket, assumindo a liderança do processo e replicando o envio de mensagens de ELECTION em direção ao Nó 4. Por possuir o maior identificador da topologia, o Nó 4 interceptou a chamada, respondeu com um pacote OK e, após o esgotamento do temporizador de confirmação sem contestações de nós maiores, autodeclarou-se o coordenador legítimo do sistema, emitindo uma mensagem do tipo COORDINATOR via broadcast para atualizar o estado de liderança em todas as instâncias ativas.

5. Fontes Utilizadas

A implementação apresentada neste trabalho foi desenvolvida de forma autoral, sem reaproveitamento direto de código de terceiros. No entanto, o código

dos três algoritmos foi orientado pelas seguintes referências teóricas e exemplos de implementação consultados durante o desenvolvimento:

- trunc8. *ricart-agrawala-algorithm* (GitHub). Disponível em: <https://github.com/trunc8/ricart-agrawala-algorithm>
- JungBok-Cho. *Bully-Algorithm* (GitHub). Disponível em: <https://github.com/JungBok-Cho/Bully-Algorithm>
- religiosa. *lamportClocks* (GitHub). Disponível em: <https://github.com/religiosa/lamportClocks>

Nenhum trecho de código dessas fontes foi diretamente copiado.